VENTION
Take-Home Assignment.

Victor Schwoertzig

17th Aprile 2026

This document has been written as part of a recruitment process for the company Vention.
Senior Robotics and Automation Developer

Assumptions on the robotic cell

- The robot is located at the global origine of the 3D environment.
- The robot base frame and the global frame are superimposed.
- The global z-axis is pointing up.
- The global X-axis is pointing toward the end-effector of the robot.
- The camera mounting pose is given with respect to the global frame.
- The pallet sits on the floor. Its global coordinate on z-axis is equal to 0.
- The robot is not equipped with a tool. The robot's flange frame is our tool frame (TCP).

The vision palletizer API offers different features.

/palletizer/configure

- Configure the pallet grid with respect to the box dimensions, pallet origin and spacing.
- From the pallet grid, compute TCP placing poses with respect to the robot frame.
- From the file "camera_detection.json", compute all TPC picking poses with respect to the robot frame.
- Send the TCP placing and picking poses to the state machine.
- Display the grid dimensions.

/palletizer/start

- Start the pick and place application. The robot will load a 2x2 pallet.
- The first motion is a moveJ to the home joint target.

/palletizer/stop

- Stop the palletizer.

/palletizer/reset

- Reset the palletizer.

/palletizer/status

- Get the current palletizer state machine status.

/palletizer/vision/detect

- Emulate the vision detection with user inputs for testing.

/palletizer/debug/positions

- Display all TCP placing poses computes in /palletizer/configuration.

/palletizer/debug/transform

- From a user pose input, return the pose with respect to the robot frame and with respect to the camera frame.

Palletizer configuration

The Figure 1 sums up our palletizer configuration: box dimensions, pallet origin, number of box (4). In the code, we defined spacing equal to 10 mm.

```
curl -X 'POST' \
  'http://localhost:8000/palletizer/configure' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "box_depth_mm": 100,
  "box_height_mm": 50,
  "box_width_mm": 100,
  "cols": 2,
  "pallet_origin_x_mm": 400,
  "pallet_origin_y_mm": -200,
  "pallet_origin_z_mm": 100,
  "rows": 2
}'
```

Figure 1: Palletizer configuration

Based on our configuration, we compute the grid dimensions as shown on Figure 2.

```
{
  "success": true,
  "message": "Positions calculated successfully",
  "grid_size": "X_grid: 110mm, Y_grid: 110mm, Layers: 1"
}
```


Figure 2: Grid dimensions

Transform coordinates

We used homogeneous transformations to compute coordinates.

The transform coordinates of an object i with respect to an object j is given by the equation (1), where H_i^j is a 4x4 homogeneous matrix, R_i^j a 3x3 rotation matrix and T_i^j a 3x1 translation vector.

$$H_i^j = \begin{bmatrix} R_i^j & T_i^j \\ 0 & 1 \end{bmatrix} \tag{1}$$

We define the robot base frame $\{R\}$, the camera frame $\{C\}$ and the detected box frame $\{B\}$ as shown on Figure 3.

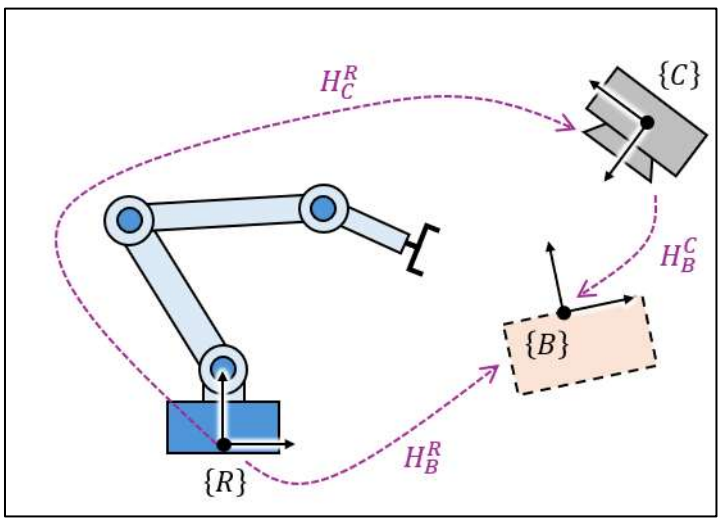

Figure 3: Transform coordinates

In our application, we implemented equation (2) in the function ***camera_to_robot()***.
The function computes the box coordinates with respect to the robot base frame, given the box coordinates with respect to the camera and the camera mounting pose with respect to the robot base frame.

$$H_B^R = H_C^R * H_B^C \tag{2}$$

We have implemented the equation (3) in the function ***robot_to_camera()***.
The function computes the box coordinates with respect to the camera frame, given the box coordinates with respect to the robot frame et the camara mounting pose with respect to the robot base frame.

$$H_B^C = (H_C^R)^{-1} * H_B^R \quad (3)$$

The equations (2) and (3) are tested in the unit test section.

TCP poses

TCP picking poses

We consider the box on the floor. The picking poses z-coordinates are equal to the box height.

Index	X (m)	Y (m)	Z (m)	Rx (rad)	Ry (rad)	Rz (rad)
0	0.557	0.293	0.050	0.262	-0.175	0.785
1	0.601	0.360	0.050	0.262	-0.175	1.047
2	0.452	0.346	0.050	0.262	-0.175	0.611
3	0.604	0.284	0.050	0.262	-0.175	0.873

Here, we think that the orientation is not correct.
We read the orientation directly from the function ***camera_to_robot()***. Supposing the box is on the floor, those results seem not logical. We will leave them here for now, knowing there is room for improvement.

TCP placing poses

We consider the default orientation when placing the box on the pallet.

Index	X (m)	Y (m)	Z (m)	Rx (rad)	Ry (rad)	Rz (rad)
0	0.400	-0.200	0.150	0	π	0
1	0.400	-0.090	0.150	0	π	0
2	0.510	-0.200	0.150	0	π	0
3	0.510	-0.090	0.150	0	π	0

Motion program

- A moveJ to the home joint target is called on the startup and at the end of the entire palletizing sequence. The home joint target is: Home_Joint = [0, - π/2, - π/2, 0, π/2, 0].
- The motion sequences are handled by the state machine.
- The approach and retracted heights are equal to 0.1m.
- The speed and acceleration are adapted: LOW during approaching and retracting; HIGH otherwise.
- A delay has been added after the gripper state changes.

TCP pose validation

The picking and placing TCP poses are validated before the cartesian motion is executed.

To validate a TCP pose, we check if it is within the robot boundaries. From the robot documentation, we found the workspace definition as shown in the Figure 4.

The workspace is a sphere with a hollow. The sphere radius is 0.85m and the hollow radius is 0.2m. We consider that, the TCP pose is not reachable if the following condition is true.

$$\sqrt{x^2 + y^2} < r_c \text{ OR } \sqrt{x^2 + y^2 + z^2} > r_s \tag{4}$$

x, y, z, the position of the tested TCP pose with respect to the robot frame.
r_c, r_s respectively the cylinder and the sphere radius.

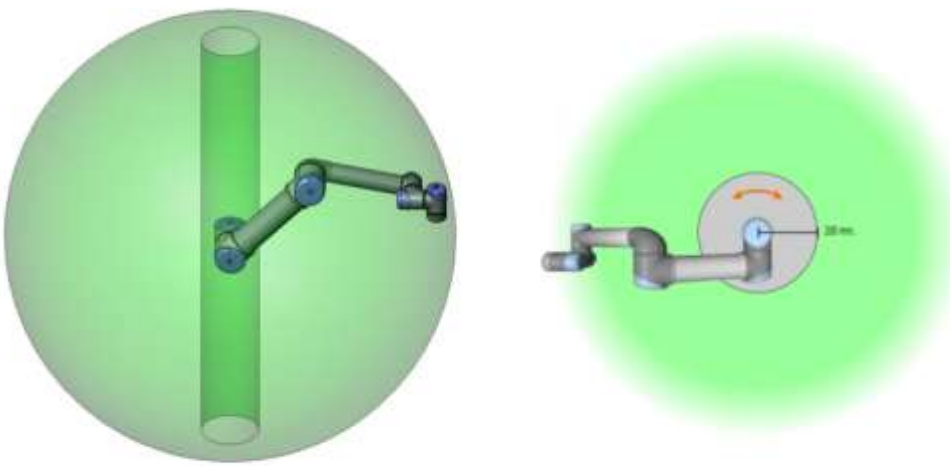

Figure 4: UR5 workspace

Simulation in MOCK

The logic has been tested in MOCK mode as shown on the Figure 5.

```
[MOCK] moveJ HOME [  0.000000,  -1.570796,  -1.570796,   0.000000,   1.570796,   0.000000]
[MOCK] moveL to [  0.556611,   0.293207,   0.150000,   0.261799,  -0.174533,   0.785398]
[MOCK] moveL to [  0.556611,   0.293207,   0.050000,   0.261799,  -0.174533,   0.785398]
[MOCK] Gripper closed
[MOCK] Waiting for 0.10 seconds...
[MOCK] moveL to [  0.556611,   0.293207,   0.150000,   0.261799,  -0.174533,   0.785398]
[MOCK] moveL to [  0.400000,  -0.200000,   0.250000,   0.000000,   3.141593,   0.000000]
[MOCK] moveL to [  0.400000,  -0.200000,   0.150000,   0.000000,   3.141593,   0.000000]
[MOCK] Gripper opened
[MOCK] Waiting for 0.10 seconds...
[MOCK] moveL to [  0.400000,  -0.200000,   0.250000,   0.000000,   3.141593,   0.000000]
Progress: {'state': 'PLACING', 'current_box': 1, 'total_boxes': 4, 'error': None}

[MOCK] moveL to [  0.600577,   0.359892,   0.150000,   0.261799,  -0.174533,   1.047198]
[MOCK] moveL to [  0.600577,   0.359892,   0.050000,   0.261799,  -0.174533,   1.047198]
[MOCK] Gripper closed
[MOCK] Waiting for 0.10 seconds...
[MOCK] moveL to [  0.600577,   0.359892,   0.150000,   0.261799,  -0.174533,   1.047198]
[MOCK] moveL to [  0.400000,  -0.090000,   0.250000,   0.000000,   3.141593,   0.000000]
[MOCK] moveL to [  0.400000,  -0.090000,   0.150000,   0.000000,   3.141593,   0.000000]
[MOCK] Gripper opened
[MOCK] Waiting for 0.10 seconds...
[MOCK] moveL to [  0.400000,  -0.090000,   0.250000,   0.000000,   3.141593,   0.000000]
Progress: {'state': 'PLACING', 'current_box': 2, 'total_boxes': 4, 'error': None}
```

```
[MOCK] moveL to [  0.452089,   0.345726,   0.150000,   0.261799,  -0.174533,   0.610865]
[MOCK] moveL to [  0.452089,   0.345726,   0.050000,   0.261799,  -0.174533,   0.610865]
[MOCK] Gripper closed
[MOCK] Waiting for 0.10 seconds...
[MOCK] moveL to [  0.452089,   0.345726,   0.150000,   0.261799,  -0.174533,   0.610865]
[MOCK] moveL to [  0.510000,  -0.200000,   0.250000,   0.000000,   3.141593,   0.000000]
[MOCK] moveL to [  0.510000,  -0.200000,   0.150000,   0.000000,   3.141593,   0.000000]
[MOCK] Gripper opened
[MOCK] Waiting for 0.10 seconds...
[MOCK] moveL to [  0.510000,  -0.200000,   0.250000,   0.000000,   3.141593,   0.000000]
Progress: {'state': 'PLACING', 'current_box': 3, 'total_boxes': 4, 'error': None}

[MOCK] moveL to [  0.603710,   0.283866,   0.150000,   0.261799,  -0.174533,   0.872665]
[MOCK] moveL to [  0.603710,   0.283866,   0.050000,   0.261799,  -0.174533,   0.872665]
[MOCK] Gripper closed
[MOCK] Waiting for 0.10 seconds...
[MOCK] moveL to [  0.603710,   0.283866,   0.150000,   0.261799,  -0.174533,   0.872665]
[MOCK] moveL to [  0.510000,  -0.090000,   0.250000,   0.000000,   3.141593,   0.000000]
[MOCK] moveL to [  0.510000,  -0.090000,   0.150000,   0.000000,   3.141593,   0.000000]
[MOCK] Gripper opened
[MOCK] Waiting for 0.10 seconds...
[MOCK] moveL to [  0.510000,  -0.090000,   0.250000,   0.000000,   3.141593,   0.000000]
Progress: {'state': 'PLACING', 'current_box': 4, 'total_boxes': 4, 'error': None}

FALL INTO: def fault
[MOCK] moveJ HOME [  0.000000,  -1.570796,  -1.570796,   0.000000,   1.570796,   0.000000]
Progress: {'state': 'FAULT', 'current_box': 4, 'total_boxes': 4, 'error': 'No more room on the pallet.'}
INFO:     172.18.0.1:52392 - "POST /palletizer/start HTTP/1.1" 200 OK
```

Figure 5: Simulation in MOCK mode

Simulation connected with URSim – Discussion on the faced issues

I encountered challenges with this section but made progress.

I successfully set up my docker – API – URSim. I identified that the robot must be switched to "Remote Mode" to enable control via the python script. In this mode, however, the URSim interface is not accessible. It prevents visualization of the robot's palletizing motion.

I explored several alternative approaches:

- Visualize with RoboDK: I was able to command the robot, but I could not access real-time joint state data from URSim.
- ROS RViz: I attempted to set up RViz in a virtual machine environment but was unable to complete the installation and configuration within the available time.
- Socket communication with URScript: I initiated this approach, but I did not have sufficient time to fully implement it.

Given more time to deepen my familiarity with URSim, and a helping hand, I am confident I could complete a full working demonstration.
In a professional setting, I would have escalated earlier and sought guidance from a supervisor or team members to unblock progress more efficiently.

Unit tests

Presentation

The purpose of the unit tests is to validate the equation (2) and (3) implementations. To do so, we create a unit test script in MATLAB to compute equations (2) and (3).

We set up the MATLAB script and the python program with the same input parameters, and we compare the results. The MATLAB script is given at the end of this section.

The camera coordinates with respect to the robot (H_C^R) are constant.

To validate the equation (2), the input values are the coordinates of the box with respect to the camera (H_B^C). The output results are the coordinates of the box with respect to the robot base frame (H_B^R).

$$H_B^R = H_C^R * H_B^C \tag{2}$$

To validate the equation (3), the input values are the coordinates of the box with respect to the robot base frame (H_B^R). The output results are the coordinates of the box with respect to the camera (H_C^R).

$$H_B^C = (H_C^R)^{-1} * H_B^R \tag{3}$$

Scenario #1

```
% Input Pose (H_in)
x_mm = 0;
y_mm = 0;
z_mm = 0;
yaw_deg = 0;
```

Command Window

```
X_Y_Z_robot =

   500   500   800

Rx_Ry_Rz_robot =

   15.0000  -10.0000   45.0000

X_Y_Z_camera =

 -691.0181 -601.6529 -374.8580

Rx_Ry_Rz_camera =

  -17.5924   -3.6929  -45.7482
```


```
{
  "Vision detection input": {
    "x_mm": 0,
    "y_mm": 0,
    "z_mm": 0,
    "yaw_deg": 0
  },
  "Input pose with respect to the Robot frame": {
    "x_mm": 500,
    "y_mm": 500,
    "z_mm": 800,
    "rx_deg": 14.999999999999998,
    "ry_deg": -10,
    "rz_deg": 45
  },
  "Input pose with respect to the Camera frame": {
    "x_mm": -691.0182640888967,
    "y_mm": -601.6529497220506,
    "z_mm": -374.8580104789365,
    "rx_deg": -17.592444552592315,
    "ry_deg": -3.692908222467993,
    "rz_deg": -45.74818255049074 6
  }
}
```


Figure 6: Unit test #1

Scenario #2

```
% Input Pose (H_in)
x_mm = 10;
y_mm = -50;
z_mm = 20;
yaw_deg = 30;
```

Command Window

```
X_Y_Z_robot =

  525.8815  255.6914  780.8456

Rx_Ry_Rz_robot =

   15.0000  -10.0000   75.0000

X_Y_Z_camera =

 -690.7691 -623.3035 -324.5470

Rx_Ry_Rz_camera =

  -17.5924   -3.6929  -15.7482
```


```
{
  "Vision detection input": {
    "x_mm": 10,
    "y_mm": -50,
    "z_mm": 20,
    "yaw_deg": 30
  },
  "Input pose with respect to the Robot frame": {
    "x_mm": 525.8815188883050,
    "y_mm": 255.69137555519994,
    "z_mm": 780.8455545670688,
    "rx_deg": 14.999999999999998,
    "ry_deg": -10,
    "rz_deg": 74.99999999999999
  },
  "Input pose with respect to the Camera frame": {
    "x_mm": -690.7691382221252,
    "y_mm": -623.3034750305787,
    "z_mm": -324.54703150040017,
    "rx_deg": -17.592444552592315,
    "ry_deg": -3.692908222467993,
    "rz_deg": -15.748182550490753
  }
}
```


Figure 7: Unit test #2

Scenario #3

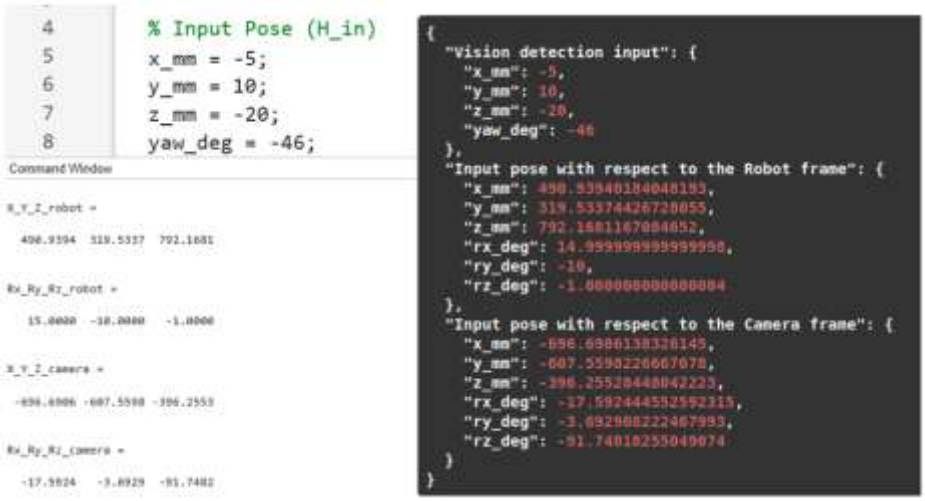

Figure 8: Unit test #3

MATLAB script

```
clear all
clc

% Input Pose (H_in)
x_mm = -5;
y_mm = 10;
z_mm = -20;
yaw_deg = -46;
H_in = get_H(x_mm, y_mm, z_mm, 0, 0, yaw_deg);

% Camera mounting pose (H_RC)
x_mm = 500;
y_mm = 300;
z_mm = 800;
roll_deg = 15;
pitch_deg = -10;
yaw_deg = 45;
H_RC = get_H(x_mm, y_mm, z_mm, roll_deg, pitch_deg, yaw_deg);


% Equation (2): camera_to_robot()
H_R = H_RC * H_in;
X_Y_Z_robot = get_position(H_R)'
Rx_Ry_Rz_robot = get_eulerAngles(H_R)

% Equation (3): robot_to_camera()
H_C = inv(H_RC) * H_in;
X_Y_Z_camera = get_position(H_C)'
Rx_Ry_Rz_camera = get_eulerAngles(H_C)


function p = get_position(H)
```

```
    p = 1000 * H(1:3, 4);
end

function eul = get_eulerAngles(H)
    R = H(1:3, 1:3);
    eul = rad2deg(rotm2eul(R, "ZYX"));
end

function H = get_H(x_mm, y_mm, z_mm, roll_deg, pitch_deg, yaw_deg)
    R = eul2rotm([deg2rad(roll_deg), deg2rad(pitch_deg), deg2rad(yaw_deg)]);
    p = 0.001 * [x_mm; y_mm; z_mm];
    H = [R, p;
         0 0 0 1];
end

function H = get_H_inv(H)
R = H(1:3, 1:3);
p = H(1:3, 4);
    H = [R', -R'*p;
         0 0 0 1];
end
```